РО 1.3. Базовое администрирование Linux

Лекция 12. Пакеты, репозитории и обновление системы

Цель лекции

- Понять, почему программы в Linux устанавливают через пакеты и репозитории.
- Научиться объяснять различие между apt, dpkg, dnf/yum и snap.
- Научиться безопасно устанавливать, обновлять, удалять и проверять пакеты.
- Понять риски сторонних репозиториев и обновлений на сервере.

Список учебных вопросов

1. Пакет как способ установки программ в Linux.
2. Репозиторий: назначение и доверенный источник пакетов.
3. Индекс пакетов и обновление списка пакетов.
4. apt: установка, удаление, обновление пакетов в Debian/Ubuntu.
5. dpkg: работа с локальными .deb-пакетами.
6. dnf/yum: пакетные менеджеры семейства RHEL на уровне сравнения.
7. snap: назначение и отличие от классических пакетов.
8. Проверка установленного пакета и версии программы.
9. Безопасность обновлений и аккуратность при подключении сторонних репозиториев.

1. Пакет как способ установки программ

Пакет - это архив с программой, служебными файлами и метаданными для установки в систему.

Метаданные описывают имя, версию, зависимости, файлы и сценарии установки.

Пакетный менеджер устанавливает программу контролируемо, а не копированием файлов вручную.

Для Debian/Ubuntu типичный формат пакета - **.deb**.

Для RHEL-подобных систем типичный формат - **.rpm**.

Что входит в пакет

- Исполняемые файлы программы.
- Конфигурационные файлы по умолчанию.
- Документация, man-страницы и служебные файлы.
- Сценарии до и после установки или удаления.
- Список зависимостей: какие библиотеки и пакеты нужны для работы.

Почему пакет лучше ручной установки

Пакетный менеджер знает, какие файлы были установлены.

Пакет можно обновить, удалить и проверить.

Зависимости устанавливаются автоматически или явно показываются администратору.

Обновления безопасности приходят из репозитория.

На сервере это снижает хаос и делает систему документируемой.

СХЕМА: ПАКЕТ, ЗАВИСИМОСТИ И УСТАНОВКА

Как работает менеджер пакетов: от запроса установки до готового приложения.

Менеджер пакетов автоматически разрешает зависимости и устанавливает всё необходимое для работы приложения.

1. ЧТО ТАКОЕ ПАКЕТ?

Пакет — это архив с программой и служебной информацией для её установки и управления.



.deb / .rpm

Содержимое пакета

- Программа (файлы)
- Библиотеки
- Метаданные
- Скрипты установки
- Информация о зависимостях

Примеры форматов:

Debian/Ubuntu → .deb (APT)

RHEL/CentOS/Fedora → .rpm (DNF/YUM)

2. КАК ПРОИСХОДИТ УСТАНОВКА ПАКЕТА?



3. ТИПЫ ЗАВИСИМОСТЕЙ

- **Зависимости (Depends)** — Обязательные пакеты, без которых программа не будет работать.
- **Рекомендуемые (Recommends)** — Улучшает функциональность, но не является обязательным.
- **Дополнительные (Suggests)** — Полезные, но необязательные дополнения.
- **Конфликты (Conflicts)** — Пакеты, которые не могут быть установлены вместе.
- **Заменяет (Provides/Obsoletes)** — Обеспечивает виртуальные пакеты или заменяет старые.

4. ЖИЗНЕННЫЙ ЦИКЛ ПАКЕТА

- **Установка (Install)** — Добавление пакета в систему.
- **Обновление (Upgrade)** — Установка новой версии пакета.
- **Настройка (Configure)** — Применение настроек и скриптов.
- **Удаление (Remove)** — Удаление пакета, но не конфигов.
- **Полное удаление (Purge)** — Удаление пакета и всех конфигов.

5. ПРИМЕР ДЕРЕВА ЗАВИСИМОСТЕЙ

Устанавливаем пакет: htop



Обозначения:

- зависит от
- необязательная связь
- уже установлено
- будет загружен
- библиотека / пакет
- служебный компонент

6. РОЛЬ РЕПОЗИТОРИЕВ

Репозиторий — хранилище пакетов и метаданных (информация о версиях, зависимостях, контрольных суммах).



Менеджер пакетов использует метаданные для поиска подходящих версий и разрешения зависимостей.

7. ПОЛЕЗНЫЕ КОМАНДЫ

Обновить списки пакетов

```
# APT (Debian/Ubuntu)
sudo apt update
# DNF (RHEL/Fedora)
sudo dnf makecache
```

Установить пакет

```
# APT
sudo apt install htop
# DNF
sudo dnf install htop
```

Показать зависимости

```
# APT
apt depends htop
# DNF
dnf deplist htop
```

Информация о пакете

```
# APT
apt show htop
# DNF
dnf info htop
```

Удалить пакет

```
# APT
sudo apt remove htop
# DNF
sudo dnf remove htop
```

Полное удаление (с конфигами)

```
# APT
sudo apt purge htop
# DNF
sudo dnf remove htop
sudo dnf autoremove
```

8. ИТОГ

- Вы запрашиваете установку пакета.
- Менеджер пакетов анализирует зависимости.
- Скачиваются и устанавливаются все необходимые пакеты.
- Скрипты настраивают систему.
- Приложение готово к использованию!

СОВЕТ Всегда обновляйте списки пакетов перед установкой и используйте официальные репозитории для безопасности и стабильности системы.

Зависимости пакетов

Зависимость - это другой пакет, без которого программа не сможет работать.

Например, веб-серверу могут понадобиться библиотеки TLS, модули и системные службы.

APT старается подобрать и установить зависимости автоматически.

Если зависимости конфликтуют, установка может быть остановлена до исправления.

Администратор должен читать список изменений перед подтверждением установки.

Пример чтения информации о пакете

Перед установкой незнакомого пакета полезно посмотреть его описание.

Для этого используют: apt show nginx.

В выводе важны поля Package, Version, Depends, Description и Download-Size.

Depends показывает зависимости, которые могут быть установлены вместе с пакетом.

Description помогает убедиться, что выбран нужный пакет.

2. Репозиторий: назначение

Репозиторий - это источник пакетов для конкретного дистрибутива и версии системы.

В репозитории хранятся пакеты, метаданные, подписи и информация о версиях.

Система получает пакеты не из случайных сайтов, а из настроенных источников.

Для сервера важно использовать доверенные и совместимые репозитории.

Доверенный источник пакетов

Доверенный репозиторий подписывает пакеты криптографическим ключом.

APT проверяет подпись индекса и пакетов, чтобы снизить риск подмены.

Если подключить случайный репозиторий, можно получить несовместимые или вредоносные пакеты.

Репозиторий должен соответствовать версии дистрибутива: например, Ubuntu 24.04 к Ubuntu 24.04.

Где описаны источники APT

Основной файл источников: /etc/apt/sources.list.

Дополнительные источники часто лежат в /etc/apt/sources.list.d/.

Современные системы могут использовать формат .sources.

После изменения источников обязательно выполняют **sudo apt update**.

Без обновления индекса APT не узнает о пакетах из нового источника.

СХЕМА: РЕПОЗИТОРИЙ, ИНДЕКС И ПАКЕТНЫЙ МЕНЕДЖЕР

Как пакеты попадают к вам в систему: от репозитория до установленного ПО.

Пакетный менеджер — это инструмент, который работает с репозиториями, чтобы находить, загружать и устанавливать пакеты вместе с их зависимостями.

1. ОБЩАЯ СХЕМА РАБОТЫ

РЕПОЗИТОРИЙ (remote)
Хранилище пакетов и метаданных на сервере.

→

ИНДЕКС (МЕТАДАННЫЕ) (index / metadata)
Списки пакетов и их версий, зависимостей, размеров, контрольных сумм.

→

ПАКЕТНЫЙ МЕНЕДЖЕР (apt, dnf, yum, pacman...)
Получает индексы, ищет пакеты, разрешает зависимости, скачивает и устанавливает.

→

КЭШ ПАКЕТОВ (cache)
Локальное хранилище скачанных пакетов и индексов.

→

УСТАНОВЛЕННЫЕ ПАКЕТЫ (system)
Файлы пакетов распакованы и настроены в системе.

2. ЧТО ТАКОЕ РЕПОЗИТОРИЙ?

Репозиторий — это сервер или зеркало, где хранятся:

- Файлы пакетов (`.deb`, `.rpm`, `.pkg.tar.zst` и др.)
- Индексы (метаданные) о пакетах
- Информация о зависимостях и версиях
- Подписи и контрольные суммы

Примеры репозиториев:

Ubuntu/Debian:	`archive.ubuntu.com`
CentOS/RHEL:	`mirror.centos.org`
Fedora:	download.fedoraproject.org
Arch Linux:	`mirror.archlinux.org`

3. ИНДЕКС (МЕТАДАННЫЕ)

Индекс — это база данных о доступных пакетах в репозитории.
Он позволяет быстро искать пакеты и проверять зависимости.

Содержит:

- Имя пакета
- Версия
- Архитектура
- Зависимости
- Размер
- Контрольные суммы
- Список файлов (опционально)

Примеры индексных файлов:

Debian/Ubuntu:	`Packages.gz`
RHEL/CentOS/Fedora:	`primary.xml.gz`
Arch Linux:	`core.db, extra.db`

4. РОЛЬ ПАКЕТНОГО МЕНЕДЖЕРА

Пакетный менеджер — это клиент, который взаимодействует с репозиториями.

Он умеет:

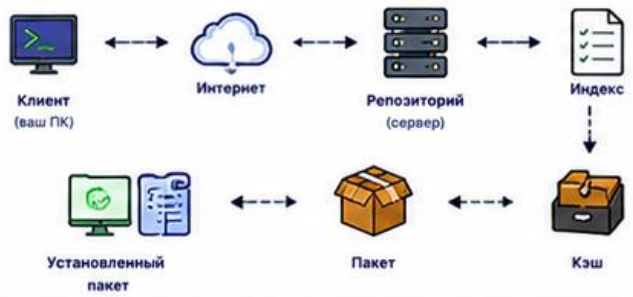
- Получать индексы
- Искать пакеты
- Разрешать зависимости
- Скачивать пакеты
- Проверять целостность
- Устанавливать, обновлять, удалять пакеты
- Работать с кэшем

Популярные менеджеры:

APT	(Debian, Ubuntu)
DNF	(Fedora)
YUM	(RHEL 7 и ниже)
Pacman	(Arch Linux)
Zypper	(openSUSE)

5. ПРИМЕР РАБОЧЕГО ПРОЦЕССА (APT)

1	Обновить списки пакетов	`sudo apt update`
2	Поиск пакета	`apt search nginx`
3	Информация о пакете	`apt show nginx`
4	Установка пакета	`sudo apt install nginx`
5	Обновление системы	`sudo apt upgrade`
6	Удаление пакета	`sudo apt remove nginx`
7	Очистка кэша	`sudo apt clean`



6. ГДЕ ХРАНЯТСЯ ДАННЫЕ (ПРИМЕР: DEBIAN/UBUNTU)

Список репозиториев	`/etc/apt/sources.list` и файлы в `/etc/apt/sources.list.d/`
Индексы (кэш)	`/var/lib/apt/lists/`
Пакеты (кэш)	`/var/cache/apt/archives/`
Установленные пакеты	`/var/lib/dpkg/status`
Конфигурация	`/etc/apt/apt.conf.d/`

7. ВАЖНО ЗНАТЬ

- Никогда не устанавливайте пакеты вручную из интернета — используйте репозитории.
- Всегда обновляйте индексы перед установкой: `sudo apt update` или `sudo dnf makecache`
- Пакетный менеджер автоматически установит необходимые зависимости.
- Используйте официальные и проверенные репозитории для безопасности и стабильности.

ПРЕИМУЩЕСТВА ТАКОГО ПОДХОДА

- Централизованное хранение и лёгкое обновление
- Гарантия целостности пакетов
- Автоматическое разрешение зависимостей
- Быстрая установка и обновление
- Безопасность за счёт подписей и проверки пакетов

СОВЕТ: Регулярно обновляйте систему: `sudo apt update && sudo apt upgrade`

Проверка источника пакета

Перед установкой полезно понять, из какого репозитория будет взят пакет.

Для этого используют: apt-cache policy nginx.

Candidate показывает версию, которую APT предлагает установить.

Таблица версий показывает источники и приоритеты.

Так можно обнаружить, что пакет приходит не из ожидаемого репозитория.

3. Индекс пакетов

Индекс пакетов - локальная база сведений о доступных пакетах и версиях.

APT не обращается к репозиторию за каждым поиском пакета.

Сначала система скачивает индекс, затем ищет пакеты локально.

Если индекс устарел, APT может не видеть новые версии или пакеты.

apt update: что делает на самом деле

apt update обновляет локальный индекс пакетов.

Эта команда не обновляет установленные программы.

Она скачивает свежие списки пакетов из настроенных репозиториев.

После apt update можно корректно искать, устанавливать и обновлять пакеты.

Пример: **sudo apt update**

Как читать результат apt update

Hit означает, что источник доступен и данные не изменились.

Get означает, что APT скачал новые данные индекса.

Err означает ошибку доступа, подписи, DNS, сети или репозитория.

Если есть ошибки репозитория, устанавливать и обновлять пакеты рискованно.

Сначала исправляют источник проблемы, затем повторяют sudo apt update.

apt list --upgradable

После обновления индекса можно посмотреть доступные обновления.

Для этого используют: **apt list --upgradable**

Команда показывает пакеты, для которых доступна новая версия.

Это безопасный подготовительный шаг перед upgrade.

На сервере список обновлений стоит просмотреть до установки.

4. apt: роль и область применения

apt - основной высокоуровневый инструмент управления пакетами в Debian/Ubuntu.

Он работает с репозиториями, зависимостями и установленными пакетами.

apt удобен для ручного администрирования в терминале.

Для сценариев иногда используют apt-get, потому что его вывод стабильнее.

В учебной практике apt достаточно для установки, удаления и обновления.

Поиск пакета через apt search

Если точное имя пакета неизвестно, используют поиск.

Пример: **apt search web server**

apt search ищет по имени и описанию пакета.

Результат нужно читать внимательно: похожие названия могут относиться к разным программам.

После поиска желательно уточнить пакет через apt show.

Просмотр пакета через apt show

apt show показывает подробную карточку пакета.

Пример: **apt show nginx**

Поля Version и Repository помогают понять источник и версию.

Depends показывает зависимости.

Description помогает понять назначение пакета до установки.

Установка пакета через apt install

Установка начинается после проверки имени и описания пакета.

Пример: **sudo apt install nginx**

APT покажет список новых пакетов, зависимостей и объем загрузки.

Перед подтверждением нужно прочитать, что именно будет установлено.

После установки выполняют отдельную проверку результата.

Проверка после установки

Проверить наличие пакета можно через:

dpkg -l | grep nginx

Проверить версию программы можно через: nginx -v.

Проверить путь к исполняемому файлу можно через:

which nginx

Если пакет устанавливает службу, проверяют:

systemctl status nginx

Так администратор подтверждает не только факт установки, но и работоспособность.

Удаление: remove и purge

apt remove удаляет пакет, но обычно оставляет конфигурационные файлы.

Пример: **sudo apt remove nginx**.

apt purge удаляет пакет вместе с конфигурацией пакета.

Пример: **sudo apt purge nginx**.

purge применяют осторожно, если настройки больше не нужны.

autoremove: очистка зависимостей

После удаления пакета могут остаться зависимости, которые больше не нужны.

Для очистки используют: **sudo apt autoremove**.

Перед подтверждением нужно прочитать список удаляемых пакетов.

На сервере нельзя автоматически соглашаться, не понимая последствий.

Если в списке важная библиотека или служба, действие нужно остановить и разобраться.

Обновление: upgrade

apt upgrade устанавливает доступные обновления установленных пакетов.

Перед ним обычно выполняют **sudo apt update**.

Пример: sudo apt upgrade.

APT покажет список обновляемых пакетов и объем загрузки.

После обновления важных компонентов может потребоваться перезагрузка.

update, upgrade и full-upgrade

apt update обновляет индекс пакетов, но не меняет программы.

apt upgrade обновляет установленные пакеты без удаления важных зависимостей.

apt full-upgrade может удалять или заменять пакеты для разрешения зависимостей.

На учебном стенде важно понимать различия, а на сервере - читать план изменений.

Для базового администрирования чаще достаточно update, list --upgradable и upgrade.

5. dpkg: назначение

dpkg - низкоуровневый инструмент работы с .deb-пакетами.

Он умеет устанавливать **локальный** .deb-файл, смотреть установленные пакеты и файлы пакета.

dpkg не скачивает зависимости из репозитория как apt.

Поэтому после dpkg -i иногда нужно исправлять зависимости через apt.

Установка локального .deb через dpkg

Локальный пакет устанавливают так:

sudo dpkg -i package.deb.

Если зависимостей не хватает, dpkg сообщит об ошибке.

После такой ошибки обычно используют:

sudo apt -f install.

apt -f install пытается установить недостающие зависимости из репозиториев.

Лучше по возможности устанавливать пакеты через apt из доверенного репозитория.

Проверка пакета через dpkg

Список установленных пакетов: **dpkg -l.**
Поиск конкретного пакета: **dpkg -l | grep nginx**.
Список файлов пакета: **dpkg -L nginx**.
Определить, какому пакету принадлежит файл:
dpkg -S /usr/sbin/nginx.
Эти команды полезны при аудите и диагностике.

СХЕМА: APT И DPKG КАК УРОВНИ УПРАВЛЕНИЯ ПАКЕТАМИ

Два уровня, одна цель — установить, обновить и удалить ПО в системе.

APT — высокий уровень: решает зависимости
DPKG — низкий уровень: устанавливает файлы

1. ДВА УРОВНЯ УПРАВЛЕНИЯ

APT (Advanced Package Tool)

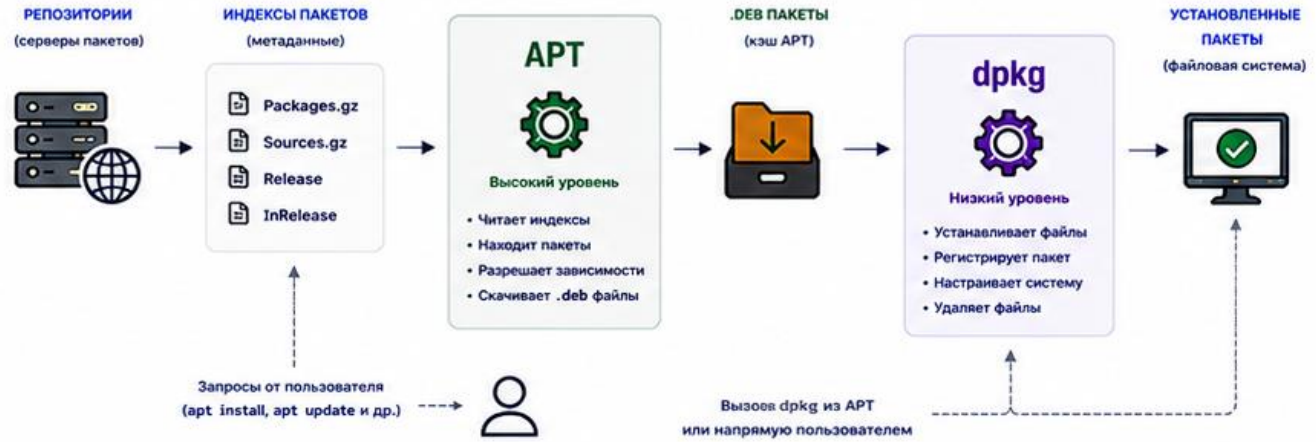
- Высокоуровневый менеджер пакетов
- Работает с репозиториями
- Разрешает зависимости
- Скачивает пакеты .deb
- Вызывает dpkg для установки

dpkg (Debian Package)

- Низкоуровневый менеджер пакетов
- Устанавливает, удаляет и настраивает .deb файлы
- Не умеет работать с репозиториями
- Не разрешает зависимости
- Работает только с уже загруженными пакетами

APT использует dpkg как движок установки.
Без dpkg система не сможет установить ни один пакет.

2. ОБЩАЯ СХЕМА ВЗАИМОДЕЙСТВИЯ



3. ЧТО ДЕЛАЕТ КАЖДЫЙ УРОВЕНЬ?

APT (высокий уровень)

- `apt update` — обновляет индексы
- `apt install <pkg>` — устанавливает пакет и все зависимости
- `apt upgrade` — обновляет установленные пакеты
- `apt remove <pkg>` — удаляет пакет (оставляя конфиги)
- `apt autoremove` — удаляет неиспользуемые зависимости

dpkg (низкий уровень)

- `dpkg -i <package.deb>` — установит пакет
- `dpkg -r <package>` — удалит пакет (без конфигов)
- `dpkg -P <package>` — удалит пакет и конфиги
- `dpkg -l` — список установленных пакетов
- `dpkg -S <file>` — какой пакет содержит файл

dpkg не скачивает пакеты и не решает зависимости.
Он только устанавливает то, что ему передали.

4. ПРИМЕР УСТАНОВКИ ПАКЕТА ЧЕРЕЗ APT

Пользователь выполняет	`apt install nginx`
APT обновляет индексы (если нужно)	читает репозитории
APT анализирует зависимости	находит и планирует установку
APT скачивает .deb пакеты	сохраняет в `/var/cache/apt/archives/`
APT вызывает dpkg	`dpkg -i package.deb`
dpkg устанавливает пакет и настраивает систему	файлы распакованы и зарегистрированы

5. ПРИМЕР УСТАНОВКИ ПАКЕТА ЧЕРЕЗ DPKG

1. Пользователь скачивает пакет вручную — `wget http://mirror/.../nginx_1.24.deb`
2. Установка напрямую через dpkg — `sudo dpkg -i nginx_1.24.deb`
3. Если есть зависимости — ошибка

```
dpkg: dependency problems prevent
configuration of nginx:
nginx depends on libc6 (>= 2.34);
  however:
  Package libc6 is not installed.
```

4. Исправление зависимостей через APT — `sudo apt -f install`

Для автоматического разрешения зависимостей используйте APT.
Для ручной установки локального .deb можно использовать dpkg.

6. ГДЕ ХРАНЯТСЯ ДАННЫЕ?

Репозитории	`/etc/apt/sources.list` `/etc/apt/sources.list.d/`
Индексы пакетов	`/var/lib/apt/lists/`
Кэш .deb пакетов	`/var/cache/apt/archives/`
База данных dpkg	`/var/lib/dpkg/status` `/var/lib/dpkg/info/`
Установленные файлы	`/` (вся файловая система)

7. СРАВНЕНИЕ

Характеристика	APT (высокий уровень)	dpkg (низкий уровень)
Работа с репозиториями	Да	Нет
Разрешение зависимостей	Да	Нет
Скачивание пакетов	Да	Нет
Установка .deb файлов	Через себя (вызывает dpkg)	Да
Удаление пакетов	Да	Да
Удобство для пользователя	Высокое	Низкое
Уровень	Высокий	Низкий

8. ПОЛЕЗНЫЕ КОМАНДЫ

APT (высокий уровень)	dpkg (низкий уровень)
`sudo apt update`	`sudo dpkg -i package.deb`
`sudo apt install <package>`	`sudo dpkg -r package`
`sudo apt upgrade`	`sudo dpkg -P package`
`sudo apt remove <package>`	`dpkg -l`
`sudo apt autoremove`	`dpkg -L package`
`sudo apt search <name>`	`dpkg -S <file>`
`apt show <package>`	`sudo dpkg --configure -a`

9. КОГДА ЧТО ИСПОЛЬЗОВАТЬ?

Используйте APT, когда:
- устанавливаете из репозиториев
- хотите, чтобы зависимости решались автоматически
- выполняете обновления системы

Используйте dpkg, когда:
- устанавливаете локальный .deb файл
- создаёте свои пакеты
- работаете на низком уровне

10. ВАЖНО ЗНАТЬ

- APT не может работать без dpkg.
- dpkg не может установить пакет без зависимостей.
- Если dpkg выдал ошибку зависимостей — используйте: `sudo apt -f install`
- Никогда не удаляйте файлы из `/var/lib/dpkg/` вручную.
- Всегда обновляйте индексы: `sudo apt update`

Не смешивайте установку вручную и через APT без необходимости — это может сломать систему.

6. dnf и yum

dnf и **yum** используются в RHEL-подобных системах: Rocky Linux, AlmaLinux, Fedora, CentOS Stream.

Они работают с RPM-пакетами и репозиториями.

yum - более старый инструмент, dnf - современная замена.

Логика похожа на apt: поиск, установка, обновление, удаление.

Синтаксис отличается, поэтому команды нельзя механически переносить между семействами Linux.

Примеры dnf/yum на уровне сравнения

- Установка пакета: sudo dnf install nginx.
- Обновление индекса и пакетов: sudo dnf upgrade.
- Информация о пакете: dnf info nginx.
- Удаление пакета: sudo dnf remove nginx.

В этом курсе dnf/yum нужны для ориентации, основной практический стенд - Debian/Ubuntu.

7. snap: назначение

snap - формат пакетов и система доставки приложений с изоляцией.

Snap-пакет часто содержит приложение и значительную часть зависимостей внутри себя.

Это упрощает установку одинаковой версии приложения на разных дистрибутивах.

Но snap может занимать больше места и вести себя иначе, чем классический deb-пакет.

snap и apt: основные отличия

- apt устанавливает deb-пакеты из репозиториев дистрибутива.
- snap устанавливает snap-пакеты из snap-store или настроенного источника.
- apt тесно связан с системой и службами дистрибутива.
- snap часто больше подходит для приложений, чем для базовых серверных служб.
- На сервере выбор формата должен быть осознанным.

Примеры snap-команд

- Список snap-пакетов: **snap list**.
- Поиск пакета: **snap find имя**.
- Установка тестового пакета: **sudo snap install hello-world**.
- Информация о пакете: **snap info hello-world**.
- Удаление: **sudo snap remove hello-world**.

8. Проверка установленного пакета и версии

Установка пакета не равна проверке результата.

Нужно проверить пакет, исполняемый файл, версию и службу, если она есть.

Разные программы используют разные ключи версии: **-v**, **--version**, **version**.

Если программа установлена как служба, нужна проверка **systemctl**.

Проверка пакета и файлов

- Проверить пакет: dpkg -l | grep nginx.
- Посмотреть файлы пакета: dpkg -L nginx.
- Найти пакет по файлу: dpkg -S /usr/sbin/nginx.
- Посмотреть источник и кандидата версии: apt-cache policy nginx.
- Эти проверки помогают документировать состояние сервера.

Проверка команды и версии

- Путь к исполняемому файлу: which nginx.
- Версия nginx: nginx -v.
- Версия OpenSSH-клиента: ssh -V.
- Версия Python: python3 --version.
- Команда версии подтверждает, что запускается ожидаемая программа.

9. Безопасность обновлений

Обновления закрывают уязвимости и исправляют ошибки.

Но обновление может изменить поведение службы или конфигурацию.

На сервере обновление планируют: проверяют список пакетов, делают резервную копию и выбирают окно обслуживания.

После обновления проверяют службы, журналы и доступность для клиентов.

Безопасный цикл обновления сервера



Ключевые правила

- Сначала backup, потом update
- Обновлять в окно обслуживания
- Проверять сервисы после обновления
- Иметь план отката

Главная идея

Безопасное обновление — это не одна команда, а цикл: подготовка, резервирование, установка, проверка и возможность отката.

Практический порядок обновления сервера

- Сначала обновить индекс: sudo apt update.
- Посмотреть доступные обновления: apt list --upgradable.
- Оценить риск: ядро, системные библиотеки, серверные службы.
- Установить обновления: sudo apt upgrade.
- После обновления проверить службы и необходимость перезагрузки.

Проверка необходимости перезагрузки

В Debian/Ubuntu индикатором часто служит файл /var/run/reboot-required.

Проверка: test -f /var/run/reboot-required && cat /var/run/reboot-required.

Если файл есть, перезагрузку нужно запланировать.

Для сервера перезагрузка выполняется в согласованное окно обслуживания.

После перезагрузки проверяют ключевые службы.

Сторонние репозитории

Сторонний репозиторий может понадобиться для свежей версии программы.

Но он может заменить системные пакеты или добавить несовместимые зависимости.

Подключать его нужно только из официального источника поставщика.

Нужно проверять ключ подписи, совместимость с версией ОС и приоритеты пакетов.

Опасный сценарий установки

Команды вида **curl URL | sudo bash** опасны, если источник не проверен.

Администратор запускает удаленный скрипт с правами root.

Нельзя заранее увидеть, какие файлы и репозитории будут изменены.

Если такой способ предлагает поставщик, нужно читать документацию и понимать каждую операцию.

В учебной работе предпочтительны штатные репозитории и понятные команды APT.

Что фиксировать в отчете

- Имя сервера и версия ОС.
- Команды apt update, apt install, apt remove или apt upgrade.
- Какие пакеты установлены, удалены или обновлены.
- Как проверен результат: версия, dpkg -l, systemctl status, журнал.
- Ошибки, предупреждения и необходимость перезагрузки.

Типовые ошибки при работе с пакетами

- Путать apt update и apt upgrade.
- Устанавливать пакет, не проверив описание и источник.
- Подтверждать autoremove, не прочитав список удаления.
- Использовать dpkg -i и не исправить зависимости.
- Подключать сторонний репозиторий без проверки доверия и совместимости.

Итоги лекции

- Пакеты позволяют устанавливать программы управляемо и проверяемо.
- Репозитории должны быть доверенными и совместимыми с версией ОС.
- apt update обновляет индекс, apt upgrade обновляет пакеты.
- dpkg полезен для локальных .deb и проверки файлов пакета.
- После установки или обновления всегда нужна отдельная проверка результата.

Контрольные вопросы

1. Что такое пакет Linux и какие данные он содержит?
2. Зачем нужен репозиторий?
3. Чем apt update отличается от apt upgrade?
4. Почему перед apt install полезно выполнить apt show?
5. Чем apt отличается от dpkg?
6. Когда может понадобиться apt -f install?
7. Чем snap отличается от классического deb-пакета?
8. Почему сторонние репозитории требуют осторожности?